

Document

CONTENTS

Streaming ASR: Partial Hypotheses, Lookahead, and Chunked Attention

Streaming LLM Generation and Early TTS Start

Streaming TTS: Time-to-First-Byte and Chunked Synthesis

Overlapping Stages: The Non-Negotiable Pattern

Barge-In Handling: Detection, Cancellation, and Echo Cancellation

Real System Architectures

Key Takeaways

Streaming ASR: Partial Hypotheses, Lookahead, and Chunked Attention

Why Whisper Is Non-Streaming

Whisper (OpenAI, 2022) uses a **full-sequence encoder-decoder** architecture with self-attention over the entire audio input. It cannot emit partial transcripts because:

- Encoder attends to all frames simultaneously (no causal masking)
- Decoder requires complete encoder output before generating first token
- No incremental decoding support; must buffer entire utterance [smallest](#)

This makes Whisper unsuitable for real-time voice agents unless wrapped in a sliding-window hack (which adds latency and loses long-range context). [smallest](#)

Streaming ASR Architectures

Modern streaming ASR systems (Deepgram Scribe v2 Realtime, Google Streaming Speech-to-Text, NVIDIA Riva) use:

1. **Chunked attention:** Encoder processes audio in overlapping 1–2 second chunks with limited right-context (e.g., 500 ms lookahead). Each chunk emits partial transcripts with confidence scores. [smallest](#)
1. **Emitting-state transducers (RNN-T):** Jointly models acoustic, transcription, and blank states; emits tokens incrementally as audio arrives without waiting for utterance end. [smallest](#)
1. **Partial hypothesis streaming:** Emits partial transcripts every 100–200 ms while user speaks. Example: Deepgram Scribe v2 Realtime returns partials in ~150 ms with PCM 8–48 kHz support. [smallest](#)

```
Audio stream → Chunked encoder → Partial transcript 1 (confidence 0.7)
                                   → Partial transcript 2 (confidence 0.85)
                                   → Final transcript (confidence 0.95)
```

Endpointing latency (deciding when user finished) is the real cost, not transcription itself. Waiting 700 ms for silence adds 700 ms to every turn. Optimizations:

- Fine-tune silence threshold to smallest value without truncating natural pauses
- Use **semantic VAD** that predicts end-of-turn from linguistic content (see prior section)
- Feed stable partials to LLM early, revise if final transcript differs (speculative execution) [smallest](#)

Streaming LLM Generation and Early TTS Start

Token-by-Token Streaming

LLMs (GPT-4, Claude, Llama 3) generate tokens autoregressively. Streaming LLM output allows TTS to begin synthesis on the **first sentence** rather than waiting for complete response. [smallest](#)

Key pattern: Buffer LLM tokens until a sentence boundary (period, question mark, exclamation point), then flush that sentence to TTS while the LLM continues generating the next sentence. [bland](#)

```
SENTENCE_END = /(?<=[.!?])\s+/

async function* speakLlmStream(tokens):
    buffer = ""
    for token in tokens:
        buffer += token
        parts = buffer.split(SENTENCE_END)
        buffer = parts.pop() ?? "" # keep incomplete fragment
        for sentence in parts:
            if sentence.trim():
                yield* synthesize(sentence.trim()) # stream to TTS
    if buffer.trim():
        yield* synthesize(buffer.trim())
```

Latency recovery: Overlapping LLM generation with TTS synthesis recovers 300–600 ms compared to batch TTS. User hears sentence 1 while LLM generates sentences 2–4. [smallest](#)

Clause-Level Chunking

Some systems (OpenAI Realtime API, LocalAI) use **clause chunking** instead of full sentences:

- Split on commas, conjunctions ("and", "but"), or length thresholds (50–80 chars)
- First chunk: 24-token limit for fast time-to-first-chunk
- Subsequent chunks: up to 96 tokens waiting for natural boundaries [github](#)

Trade-off: Smaller chunks reach player sooner (lower first-byte latency) but risk choppy prosody. Larger chunks sound more natural but delay first audio. Production systems bias toward smaller chunks early in the utterance (user is waiting) and larger chunks later (audio already playing). [elevenlabs](#)

Streaming TTS: Time-to-First-Byte and Chunked Synthesis

Time-to-First-Byte (TTFB) Architecture

TTFB measures time from text arriving at TTS API to first audio byte returned. Modern streaming TTS (ElevenLabs Flash v2.5, Deepgram Aura-2, Cartesia Sonic) achieve 75–120 ms TTFB for short inputs. [bland](#)

How it works:

- Predict-ahead mechanism:** TTS model begins phoneme synthesis from first few tokens using lookahead (e.g., 120 chars = ~75 ms on Flash v2.5) [usesherlock](#)
- WebSocket streaming:** Persistent bi-directional connection eliminates per-request handshake overhead; audio chunks flow back as generated [bland](#)
- Chunked transfer encoding:** HTTP/1.1 chunked responses or WebSocket frames deliver audio incrementally; playback starts before synthesis completes [bland](#)

Sentence/Clause Chunking Strategies

STRATEGY	CHUNK SIZE	FIRST-BYTE LATENCY	PROSODY QUALITY	USE CASE
Sentence boundary	10–50 words	100–200 ms	Natural (complete intonation)	Default for most agents elevenlabs
Clause boundary	5–15 words	75–150 ms	Slightly choppy	Low-latency agents localai
Fixed token count	24–96 tokens	50–100 ms	Robotic (mid-clause breaks)	Experimental; not recommended github

Optimization steps:

- Use streaming WebSocket TTS endpoint, not synchronous HTTP [callsphere](#)
- Set `chunk_length_schedule` first value to 50–80 chars for sub-100 ms TTFB [callsphere](#)
- Stream LLM output into TTS at sentence boundaries—do not wait for full reply [elevenlabs](#)

Player Buffer Tuning

Audio players buffer 100–500 ms by default to avoid stuttering on network jitter. This buffer adds directly to perceived latency. [elevenlabs](#)

CONNECTION TYPE	RECOMMENDED BUFFER	RATIONALE
Stable (co-located server)	50–150 ms	Low jitter risk; shaves 200+ ms off TTFA elevenlabs
Mobile/cross-region	200–400 ms	Prevents audible gaps; jitter outweighs latency cost elevenlabs

Overlapping Stages: The Non-Negotiable Pattern

Sequential pipeline (no overlap):

```
User speaks → VAD → ASR (complete) → LLM (complete) → TTS (complete) → Playback
TTFA = ASR + LLM_TTFT + TTS_TTFB + playback_buffer = 1,200–2,000 ms
```

Overlapping pipeline (streaming everywhere):

```
User speaks → VAD → ASR partials → LLM starts on stable partials → TTS starts on
TTFA = endpointing + LLM_TTFT_partial + TTS_TTFB = 400–700 ms
```

Key overlaps:

- **ASR → LLM:** Feed stable partials (confidence >0.85) to LLM before user finishes; revise if final transcript differs [smallest](#)
- **LLM → TTS:** Flush sentence 1 to TTS while LLM generates sentences 2–4 [bland](#)
- **TTS → Playback:** Start playback on first audio chunk; trim player buffer to minimum safe value [elevenlabs](#)

Latency budget example (well-optimized 2026 system): [smallest](#)

- VAD + audio capture: 50 ms

- ASR transcription: 150 ms
- LLM time-to-first-token: 400 ms
- TTS first chunk: 150 ms
- Network overhead: 50 ms
- **Total TTFA:** 800 ms (sub-500 ms achievable with aggressive tuning) [smallest](#)

Barge-In Handling: Detection, Cancellation, and Echo Cancellation

Detecting User Speech During Playback

Barge-in (interruption handling) allows users to speak while the agent is talking, causing the agent to stop and process new input. This requires:

1. **Continuous VAD monitoring:** Run VAD on microphone input even during agent playback. Neural VAD (Silero v5) detects speech onset in <20 ms with 6.8% false-positive rate.
1. **Acoustic Echo Cancellation (AEC):** Remove agent's audio from microphone input to prevent self-triggering. NLMS (Normalized Least Mean Squares) adaptive filters subtract the known playback signal from the mic stream. [smallest](#)

```
Mic input → AEC (subtract speaker output) → VAD → Barge-in detection
```

Cancelling In-Flight Generation

When barge-in is detected:

1. **Flush audio buffer:** Immediately stop TTS playback; discard any buffered audio chunks not yet played [smallest](#)
2. **Cancel LLM generation:** Abort in-progress token generation; clear KV cache to free memory [smallest](#)
3. **Restart ASR pipeline:** Reset streaming ASR state; begin transcribing new utterance without losing initial frames [smallest](#)

State machine example (Multigrid SDK):

```
State: SPEAKING
  on barge-in detected → flush_audio_buffer()
                        cancel_llm_generation()
                        reset_asr_stream()
                        transition to LISTENING
```

Full-Duplex vs. Half-Duplex

- **Half-duplex** (traditional): Agent must finish speaking before listening; barge-in requires explicit cancellation logic [smallest](#)
- **Full-duplex** (Moshi, OpenAI Realtime): Two simultaneous audio streams (user + agent); model processes both concurrently without turn-taking mechanism

Trade-off: Full-duplex enables natural overlap (backchannels, interruptions) but complicates tool calling and guardrails mid-turn. Most production systems use half-duplex with aggressive barge-in detection. [smallest](#)

Real System Architectures

OpenAI Realtime API

- **Streaming ASR:** Partial transcripts every 100–200 ms; semantic VAD for endpointing
- **Streaming LLM:** Token-by-token generation; function calling mid-stream
- **Streaming TTS:** Clause-level chunking; 75–150 ms TTFB with Flash model [callsphere](#)
- **Barge-in:** Continuous VAD + AEC; cancels in-flight generation on speech onset

ElevenLabs + LLM Pipeline

- **ASR:** Deepgram Scribe v2 Realtime (150 ms partials) [smallest](#)
- **LLM:** Stream tokens to TTS at sentence boundaries [elevenlabs](#)
- **TTS:** Flash v2.5 with WebSocket streaming; 75 ms inference + 50–100 ms network = 125–175 ms TTFB [bland](#)

- **Chunking:** First chunk 24 tokens; subsequent chunks 96 tokens [github](#)

Deepgram Aura Pipeline

- **Integrated STT + TTS:** Single WebSocket connection for both directions; co-located inference reduces network hops [bland](#)
- **Semantic endpointing:** Multi-horizon inactivity prediction (0.5s, 1s, 2s, 3s)
- **TTFB:** <200 ms for Aura-2; sentence-level chunking for natural prosody [lukeocodes](#)

Bland.ai Co-Located Stack

- **Voice model + inference + delivery:** Single pipeline eliminates inter-service round trips [bland](#)
- **Sub-400 ms TTFA:** Achieved by co-locating all layers; no third-party TTS hop [bland](#)
- **Telephony-optimized:** μ -law 8 kHz codec matches PSTN; no transcoding latency [bland](#)

Key Takeaways

1. **Stream everywhere:** ASR partials → LLM tokens → TTS chunks; overlap stages to recover 300–600 ms [smallest](#)
2. **Chunk at sentence boundaries:** Balance first-byte latency (smaller chunks) vs. prosody quality (larger chunks) [elevenlabs](#)
3. **Tune endpointing aggressively:** Silence threshold 300–500 ms for snappy agents; use semantic VAD to reduce dead air
4. **Barge-in requires AEC:** Subtract agent audio from mic input; flush buffers and cancel generation on speech onset [smallest](#)
5. **Measure P50 and P95 TTFA:** Tail latency (P95) is what users feel; benchmark per region against your deployment [bland](#)

Production systems in 2026 target **P50 TTFA <800 ms, P95 <1.5 s** for natural conversational feel. Aggressive streaming and overlap optimizations are non-negotiable for sub-second latency. [smallest](#)

